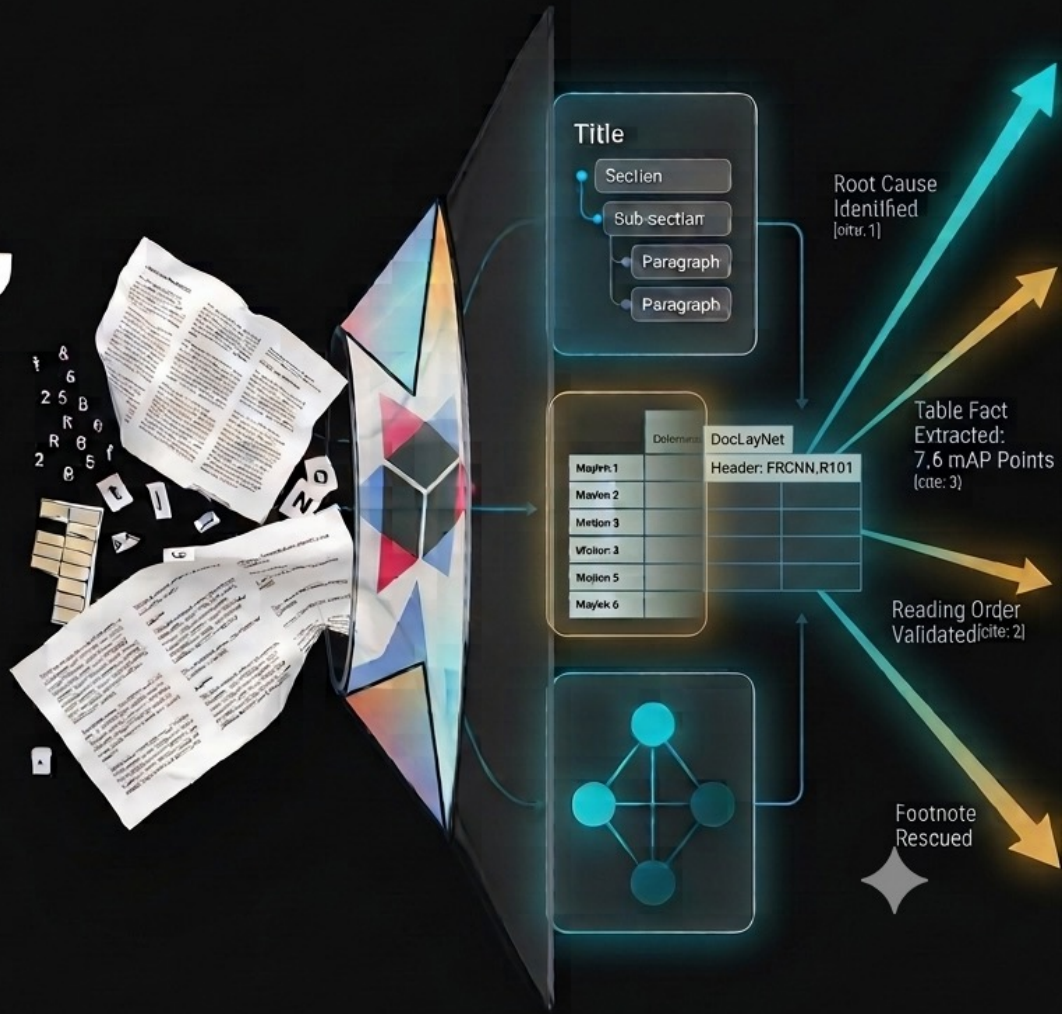
Garbage In, Insight Out

Document Intelligence for AI-Infused Java Applications

Eric Deandrea • Java Champion



next dev

Who wrote this paper?

"...the paper titled **"Segmentation for document layout analysis: not dead yet"** was written by Logan Markewich, Hao Zhang, Yubin Xing, Navid Lambert-Shirzad, Jiang Zhexin, Roy Lee, Zhi Li, and Seok-Bum Ko."



Who am I?



- Java Champion
- 27+ years software development experience
- Works on Open Source projects
  - Quarkus
  - LangChain4j, Quarkus LangChain4j
  - Docling Java, Quarkus Docling
  - Langfuse Java, Quarkus Langfuse
  - Spring Boot, Spring Framework, Spring Security
  - Testcontainers
  - Wiremock
  - Microcks
- Boston Java Users ACM Chapter Vice Chair & Board Member
- Published Author
- Cat lover

IBM

Sessionize MOST ACTIVE SPEAKER 2025

@edeandrea

Quick Primer



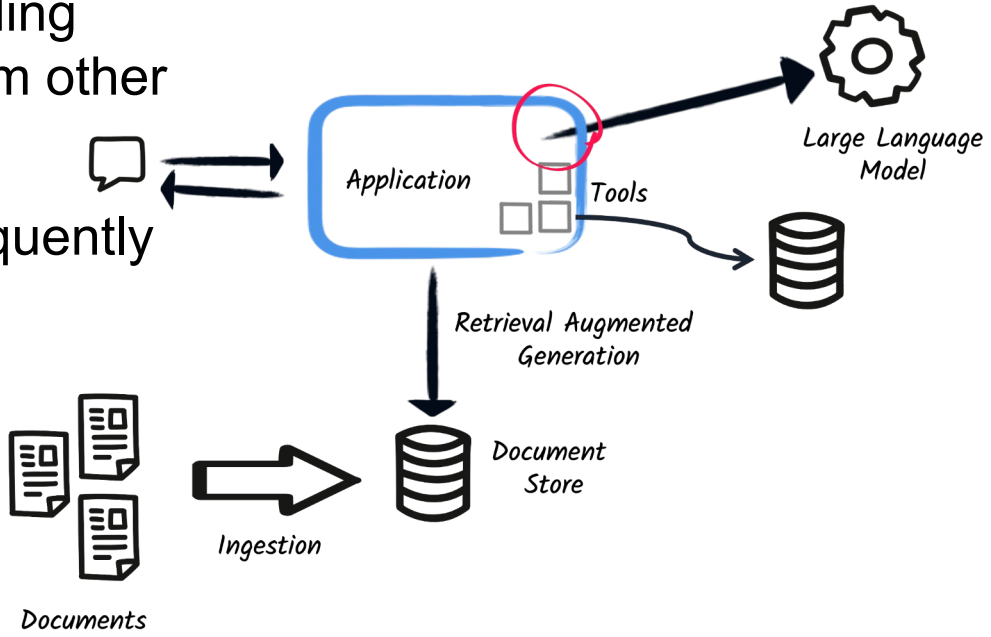
Retrieval Augmented Generation (RAG)

Enhance LLM knowledge by providing relevant information in real-time from other sources

- Dynamic data that changes frequently
  - Fine-tuning is expensive!

2 stages

- Indexing / Ingestion
- Retrieval / Augmentation



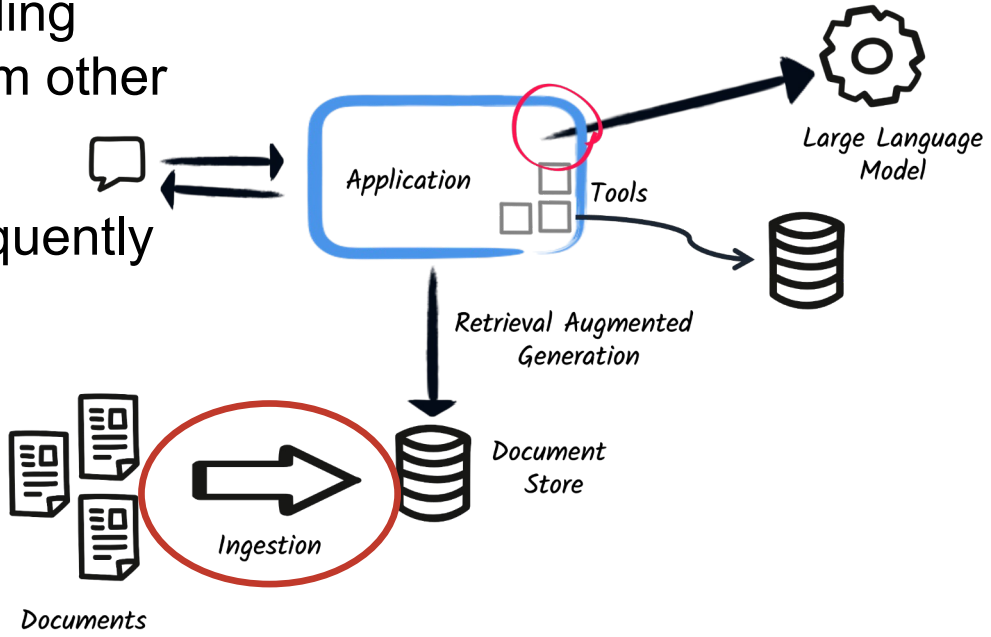
Retrieval Augmented Generation (RAG)

Enhance LLM knowledge by providing relevant information in real-time from other sources

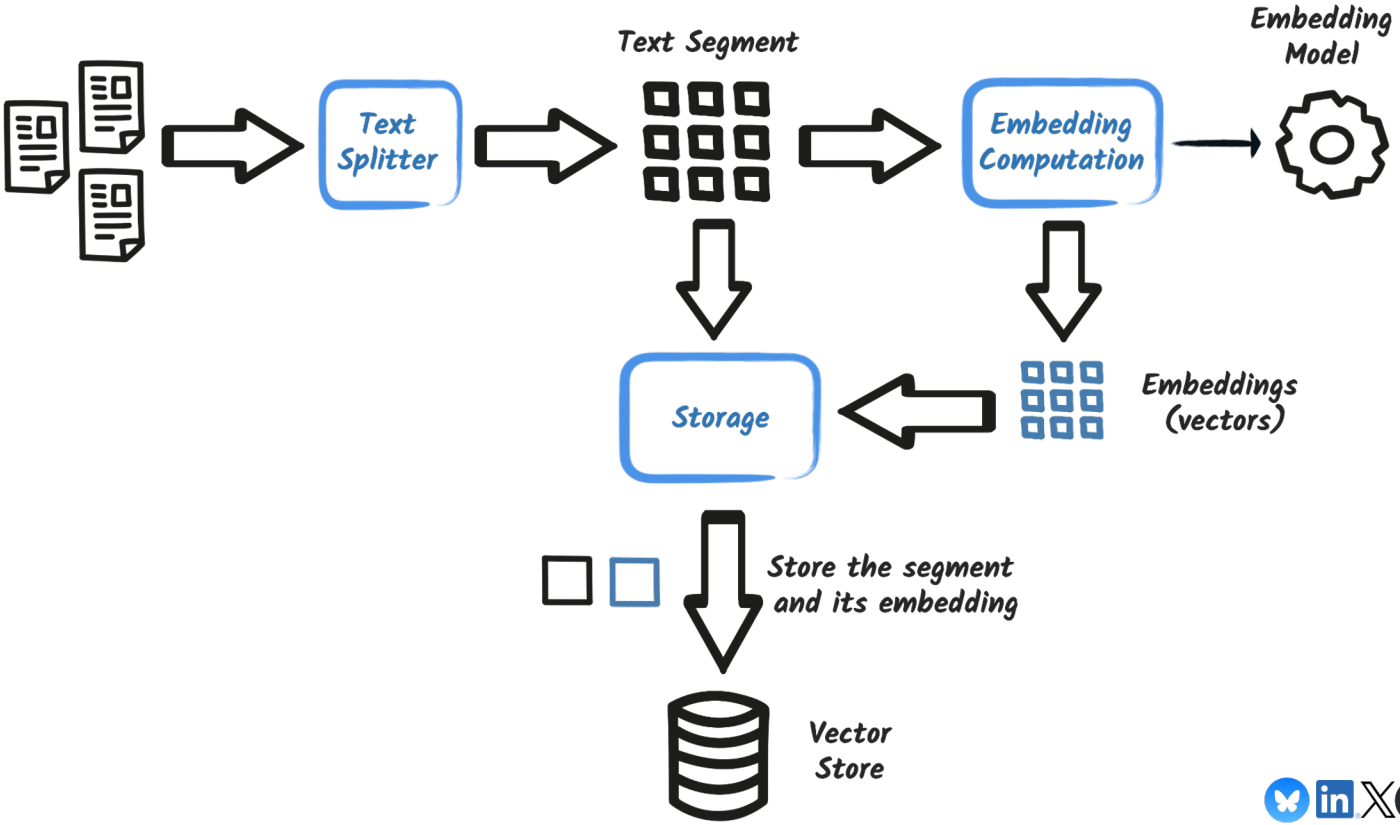
- Dynamic data that changes frequently
  - Fine-tuning is expensive!

2 stages

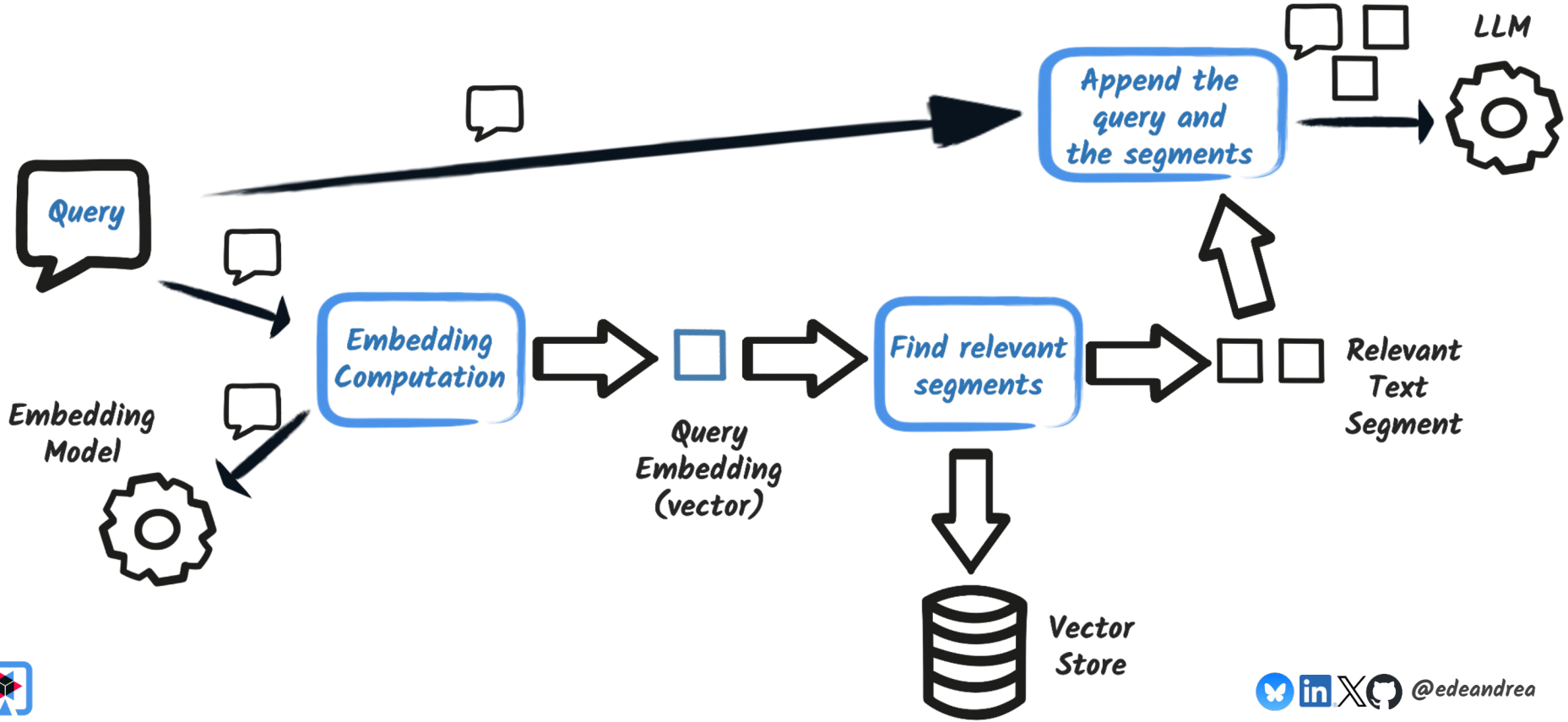
- Indexing / Ingestion
- Retrieval / Augmentation

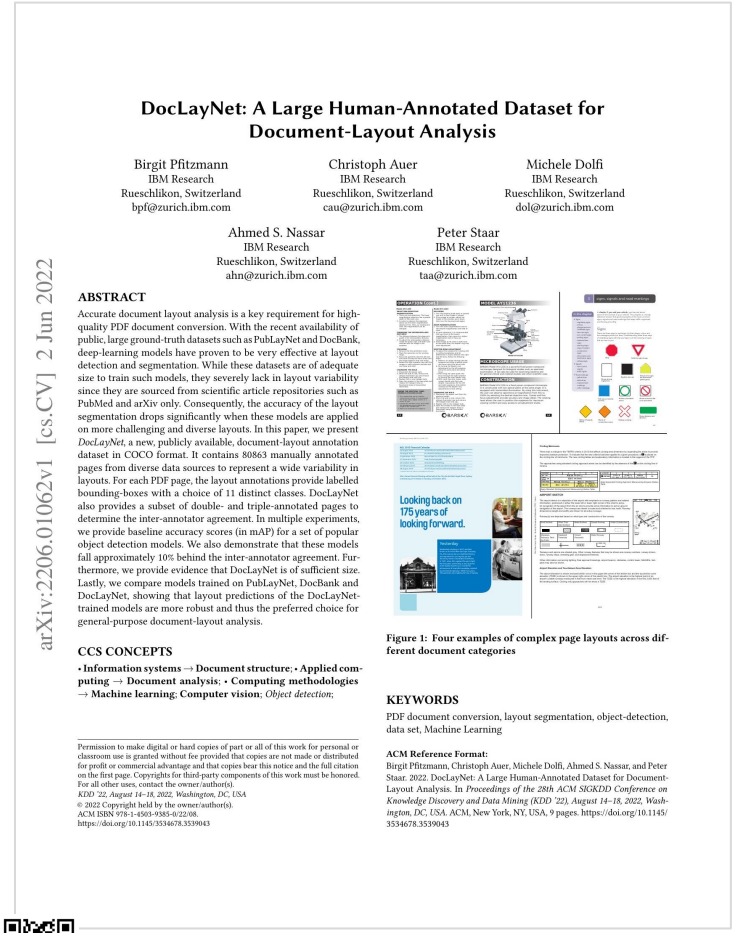


Indexing / Ingestion



Retrieval / Augmentation





DocLayNet: A Large Human-Annotated Dataset for Document-Layout Analysis

Birgit Pfitzmann
IBM Research
Rueschlikon, Switzerland
bpf@zurich.ibm.com

Christoph Auer
IBM Research
Rueschlikon, Switzerland
cau@zurich.ibm.com

Michele Dolfi
IBM Research
Rueschlikon, Switzerland
dol@zurich.ibm.com

Ahmed S. Nassar
IBM Research
Rueschlikon, Switzerland
ahn@zurich.ibm.com

Peter Staar
IBM Research
Rueschlikon, Switzerland
taa@zurich.ibm.com

arXiv:2206.01062v1 [cs.CV] 2 Jun 2022

ABSTRACT

Accurate document layout analysis is a key requirement for high-quality PDF document conversion. With the recent availability of public, large ground-truth datasets such as PubLayNet and DocBank, deep-learning models have proven to be very effective at layout detection and segmentation. While these datasets are of adequate size to train such models, they severely lack in layout variability since they are sourced from scientific article repositories such as PubMed and arXiv only. Consequently, the accuracy of the layout segmentation drops significantly when these models are applied on more challenging and diverse layouts. In this paper, we present *DocLayNet*, a new, publicly available, document-layout annotation dataset in COCO format. It contains 80863 manually annotated pages from diverse data sources to represent a wide variability in layouts. For each PDF page, the layout annotations provide labelled bounding-boxes with a choice of 11 distinct classes. DocLayNet also provides a subset of double- and triple-annotated pages to determine the inter-annotator agreement. In multiple experiments, we provide baseline accuracy scores (in mAP) for a set of popular object detection models. We also demonstrate that these models fall approximately 10% behind the inter-annotator agreement. Furthermore, we provide evidence that DocLayNet is of sufficient size. Lastly, we compare models trained on PubLayNet, DocBank and DocLayNet, showing that layout predictions of the DocLayNet-trained models are more robust and thus the preferred choice for general-purpose document-layout analysis.

CCS CONCEPTS

• **Information systems** → **Document structure**; • **Applied computing** → **Document analysis**; • **Computing methodologies** → **Machine learning**; **Computer vision**; *Object detection*;

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
KDD '22, August 14–18, 2022, Washington, DC, USA
© 2022 Copyright held by the owner/author(s).
ACM ISBN 978-1-4503-9385-0/22/08.
https://doi.org/10.1145/3534678.3539043

Figure 1: Four examples of complex page layouts across different document categories

KEYWORDS

PDF document conversion, layout segmentation, object-detection, data set, Machine Learning

ACM Reference Format:
Birgit Pfitzmann, Christoph Auer, Michele Dolfi, Ahmed S. Nassar, and Peter Staar. 2022. DocLayNet: A Large Human-Annotated Dataset for Document-Layout Analysis. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '22), August 14–18, 2022, Washington, DC, USA.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3534678.3539043

DocLayNet

Pfitzmann, Auer, Dolfi, Nassar, Staar (IBM Research, KDD 2022)

A real, peer-reviewed paper. Five-author header, dense two-column body, complex tables, figures, footnotes: everything our abstract complains about.

It's a paper about document-layout analysis. It's about to have its own layout analyzed.

arxiv.org/abs/2206.01062

@edeandrea

Act 1: Mode A

The obvious way



Mode A: The obvious way

Extract

Apache Tika: plain text, no layout awareness

Chunk

Sentence splitter: fixed rules, no table sense

This is the same code most people reach for first. This is what it actually does to our document.



LIVE

Mode A: “Who wrote the paper?”

“...the paper titled **“Segmentation for document layout analysis: not dead yet”** was written by Logan Markewich, Hao Zhang, Yubin Xing, Navid Lambert-Shirzad, Jiang Zhexin, Roy Lee, Zhi Li, and Seok-Bum Ko.”

“This is the most clearly identified paper in the context with its author list provided.”



LIVE

Mode A: “Who wrote the paper?”

“...the paper titled **“Segmentation for document layout analysis: not dead yet”** was written by Logan Markewich, Hao Zhang, Yubin Xing, Navid Lambert-Shirzad, Jiang Zuoxin, Roy Lee, Zhi Li, and Seok-Bum Ko.”

“This is the most clearly identified paper in the context with its author list provided.”

That's citation [5]. Not this paper.



LIVE

Mode A: "Who wrote the paper?"

"...the paper titled **"Segmentation for document layout analysis: not dead yet"** was written by Logan Markewich, Hao Zhang, Yubin Xing, Navid Lambert-Shirzad, Jiang Zhexin, Roy Lee, Zhi Li, and Seok-Bum Ko."

"This is the most clearly identified paper in the context with its author list provided."

That's citation [5]. Not this paper.

What actually got retrieved (top 4, by score):

0.80 "In 2017 14th IAPR International Conference on Document Analysis and Recognition …"

0.79 "In Proceedings of the 28th International Conference on Computational Linguistics…"

0.79 "In Proceedings of the International Conference on Document Analysis and Recognit…"

0.78 "In Proceedings of the International Conference on Document Analysis and Recognit…"



Mode A: The Extractor

```
package dev.ericdeandrea.docling.ai.ingestion.extraction;

public class TikaExtractor {
  private final ApacheTikaDocumentParser parser = new ApacheTikaDocumentParser();

  public ExtractionResult extract(Path documentPath) {
    return new ExtractionResult(FileSystemDocumentLoader.loadDocument(documentPath, parser));
  }
}
```

The extraction logic, imports omitted. No configuration, no tuning, no reason to suspect it's wrong.



Mode A: The Chunker

```
package dev.ericdeandrea.docling.ai.ingestion.chunking;

public class NaiveChunker {

  public List<TextSegment> chunk(ExtractionResult result, Mode mode) {
    var splitter = new DocumentBySentenceSplitter(maxTokens, overlap);
    var segments = splitter.split(result.document());

    // 2 sentences before + after, joined as "extended_content" metadata
    enrichWithSurroundingContext(segments, 2, 2);

    return segments.stream()
      // attachMetadata() tries to attach provenance, but there isn't any here.
      .map(segment -> attachMetadata(segment, result, mode))
      .toList();
  }
}
```



Retrieval: the boring part, on purpose

```java
package dev.ericdeandrea.docling.ai.rag;

final class ExtendedContentRetrievalAugmentorBuilder {

  static RetrievalAugmentor build(EmbeddingStore<TextSegment> embeddingStore,
                                  EmbeddingModel embeddingModel, DemoConfig config) {
    var retriever = EmbeddingStoreContentRetriever.builder()
      .embeddingStore(embeddingStore)
      .embeddingModel(embeddingModel)
      .maxResults(config.rag().topK())
      .build();

    // This is the line that surfaces the chunker's "extended_content"
    // enrichment to the LLM. Everything else here is default wiring.
    var contentInjector = DefaultContentInjector.builder()
      .metadataKeysToInclude(List.of("extended_content"))
      .build();

    return DefaultRetrievalAugmentor.builder()
      .contentRetriever(retriever)
      .contentInjector(contentInjector)
      .build();
  }
}
```

Same for all three modes. Mostly LangChain4j's own builder vocabulary, barely any of it is ours.



“vegetative electron microscopy”

A phrase that means nothing. It's appeared in dozens of papers anyway.

It's now used as a fingerprint by paper-mill screening tools.

One theory, not confirmed: a column-misread spliced two phrases together.

Reading order isn't just a demo inconvenience. It's already real.



retractionwatch.com/2025/02/10/vegetative-electron-microscopy-fingerprint-paper-mill

Why not just a bigger model?

Cost

Running a vision-language model over every page doesn't scale the way a purpose-built extraction step does.



Why not just a bigger model?

Cost

Running a vision-language model over every page doesn't scale the way a purpose-built extraction step does.

Latency

Big vision models are slow. Fine for a demo, not fine for a pipeline processing real volume.



Why not just a bigger model?

Cost

Running a vision-language model over every page doesn't scale the way a purpose-built extraction step does.

Latency

Big vision models are slow. Fine for a demo, not fine for a pipeline processing real volume.

Capacity

Throwing the whole document at a bigger model just moves the chunking problem earlier. It doesn't remove it.



Why not just a bigger model?

Cost

Running a vision-language model over every page doesn't scale the way a purpose-built extraction step does.

Latency

Big vision models are slow. Fine for a demo, not fine for a pipeline processing real volume.

Capacity

Throwing the whole document at a bigger model just moves the chunking problem earlier. It doesn't remove it.

Privacy

A lot of enterprises can't send proprietary documents to a third-party API. On-prem isn't optional for them.



Act 2: Mode B

Enter Docling



Meet Docling

Open source
MIT license, Linux Foundation project

Document intelligence
Advanced, cost-efficient

Multi-format
PDF, Word, PowerPoint, HTML, and more

Unified representation
Rich, LLM-friendly

Privacy & security
Self-host it, or use the managed service

Plug-and-play
For building apps & agents





docling.ai















docling-project.github.io/docling-java



Mode B: One thing changes

Extract

Docling instead of Tika: structured extraction, layout & table awareness.

Chunk

The exact same sentence splitter as Mode A. Unchanged. On purpose.

Everything downstream of extraction (embedding model, vector store, retrieval, prompt, LLM) stays identical to Mode A.



Mode B: One thing changes

Extract

Docling instead of Tika: structured extraction, layout & table aware.

Chunk

The exact same sentence splitter as Mode A. Unchanged. On purpose.

Everything downstream of extraction (embedding model, vector store, retrieval, prompt, LLM) stays identical to Mode A.

If the answer changes, extraction is the only thing that could have caused it.



LIVE

“Who wrote the paper?” again

Mode A (Tika)

“...the paper titled **“Segmentation for document layout analysis: not dead yet”** was written by Logan Markewich, Hao Zhang, Yubin Xing, Navid Lambert-Shirzad, Jiang Zhexin, Roy Lee, Zhi Li, and Seok-Bum Ko.”

Wrong. Still.

Mode B (Docling)

“...the paper “DocLayNet: A Large Human-Annotated Dataset for Document-Layout Analysis” was written by: Birgit Pfitzmann, Christoph Auer, Ahmed S. Nassar, Michele Dolfi, and Peter Staar”

...then confirms it by walking through every author's IBM Research Rueschlikon address and email from the extended-content block, plus the arXiv:2206.01062v1 metadata.

Correct. Verbose.



Mode B: The Extractor Swap

```
package dev.ericdeandrea.docling.ai.ingestion.extraction;

public class DoclingNaiveExtractor {

  public Uni<ExtractionResult> extract(Path documentPath) {
    var parser = DoclingDocumentParser.builder()
        .doclingClient(this.doclingServeApi)
        .documentRequest(request)
        .documentExtractor(response -> {
         // Flattens the structured DoclingDocument into the full-text Document
         // The naive chunker sentence-splits
          var doclingDoc = response.getDocument().getJsonContent();
          doclingDocumentHolder.set(doclingDoc);

          return Document.from(buildFullText(doclingDoc));
        })
        .build();

    return Uni.createFrom()
        .completionStage(Unchecked.supplier(() -> {
          var in = Files.newInputStream(documentPath);
          return parser.parseAsync(in)
              .whenComplete((_, _) -> closeQuietly(in));
        }))
        .map(document -> new ExtractionResult(
            document, buildProvenance(doclingDocumentHolder.get(), document.text())));
  }
}
```



buildFullText() and buildProvenance() do the real work here.

Act 3: Mode C

How far this goes



Mode C: One more thing changes

Extract + chunk

Docling's hybrid chunker, in one server-side round trip. No local NaiveChunker at all this time.

Table rows become facts

Each cell keeps its row and column label attached, e.g. “All, FRCNN.R101 = 73.4”, so one chunk carries a complete fact instead of needing to correlate headers across chunks.

Orphaned figure text gets rescued

Chart labels that would otherwise be silently dropped get collected into their own retrievable chunk.



LIVE

Full circle: “Who wrote the paper?”

Mode A

“...**Segmentation for document layout analysis: not dead yet**… Logan Markewich, Hao Zhang, et al.”

Wrong.

Mode B

“...the paper … was written by: Birgit Pfitzmann, Christoph Auer, Ahmed S. Nassar, Michele Dolfi, and Peter Staar”

...then walks through every author's IBM Research address and email from the extended-content block, plus the arXiv metadata, to confirm it.

Correct. Verbose.

Mode C

“...the authors of the paper are: Birgit Pfitzmann, Christoph Auer, Michele Dolfi, Ahmed S. Nassar, and Peter Staar”

“...clearly stated in the ACM Reference Format section at the beginning of the document.”

Correct. Direct.



LIVE

The easy questions tie too

“How many annotators were used in the production annotation phase, and how long did it take?”

A, B, and C: 32 annotators, approximately three months. All three, correct.

“What percentage of DocLayNet pages are Patents?”

A, B, and C: 8%. All three, correct. (Mode C only gets there because of the orphaned-picture rescue from the last slide.)



LIVE

Same answer, wildly different evidence

“What does Table 2 show, and what network architecture won overall?”

Mode A

“YOLOv5x won overall.”

Correct. Bare assertion, no scores, no metadata.

Mode B

“YOLOv5x6 won overall in Table 2.”

Correct. Same assertion, now with page/type metadata in the chunks.

Mode C

YOLOv5x6: 77.7, R101: 71.5, FRCNN: 70.1, R50: 68.4

Correct, with the actual scores.



LIVE

Only Mode C can answer

“By how many mAP points does YOLOv5x outperform Faster R-CNN overall?”

Mode A

“I don't have enough information.”

Mode B

“The actual table with numerical values is not included in the provided context.”

Mode C

“FRCNN.R101: 70.1, YOLOv5x6: 77.7. The difference is 77.7 - 70.1 = **7.6 mAP points.**”



Mode C: extract and chunk, one call

```
package dev.ericdeandrea.docling.ai.ingestion.extraction;

public class DoclingHybridExtractor {

  public Uni<List<TextSegment>> extractAndChunk(Path documentPath) {
    // One request: extraction AND structure-aware chunking, server-side.
    var request = HybridChunkDocumentRequest.builder()
      .includeConvertedDoc(true).build();

    doclingServeApi.chunkFilesWithHybridChunkerAsync(request, documentPath)
      .map(response -> {
       // Server already returns fully-formed chunks. No local chunker.
        var segments = response.getChunks().stream()
          .map(chunk -> TextSegment.from(chunk.getText(), metadataFor(chunk)))
          .collect(toCollection(ArrayList::new));

       // Rescue chart labels the chunker would otherwise drop.
        rescueOrphanedPictureText(response, segments);
        return segments;
      });
  }
}
```

Simplified for the slide: the real method inlines this logic rather than calling named helpers. Same behavior.



Not just a Quarkus thing



LangChain4j

What this whole demo runs on

Quarkus

quarkus-docling, co-maintained with Alex Soto

Spring Boot

Arconia by Thomas Vitale, plugs into Spring AI's RAG pipeline

*Thomas Vitale and I built the same idea independently, then joined forces: **docling-java**, one foundation under all three.*

Takeaways

What we actually proved



Five questions, three modes, all real

Question	Mode A	Mode B	Mode C
Annotators & timeline	✓	✓	✓
Patents percentage	✓	✓	✓
Who wrote the paper?	✗	✓	✓
Table 2 winner *(bare → metadata → scores)*	✓	✓	✓✓
mAP difference *(only C answers)*	✗	✗	✓✓



✗ couldn't ✓ answered ✓✓ answered with grounded evidence (page / table / scores)

Thank You!

Demo app source

github.com/edeandrea/garbage-in-insight-out

These slides

bit.ly/4jKvnOx

@edeandrea